

Progress Report

Day 15 — Authentication Forms and State Management

Task Management MERN Stack Platform

Shafin Nigamana | 03 June 2026

1. Day-15 Objective

The objective of Day 15 was to implement frontend authentication functionality, establish centralized authentication state management, and integrate the React application with backend authentication APIs.

2. Tasks Completed

Login Form Development

- Implemented a user login interface using React state management.

Configured fields:

- Email
- Password

Implemented features:

- Required field validation
- Loading state handling
- Error message display
- Backend authentication integration

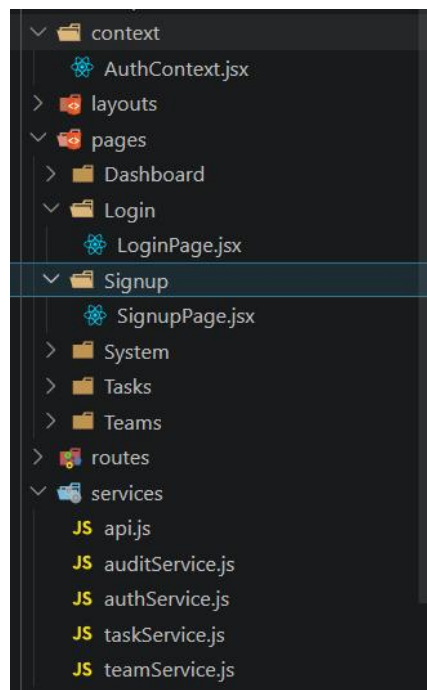


Figure 2.1 — Authentication-related frontend modules created and configured.

Signup Form Development

- Developed a user registration interface connected to backend authentication services.

Configured fields:

- Full Name
- Email
- Password
- Account Role

Implemented features:

- Form validation
- Loading state handling
- Error handling
- Successful registration redirection

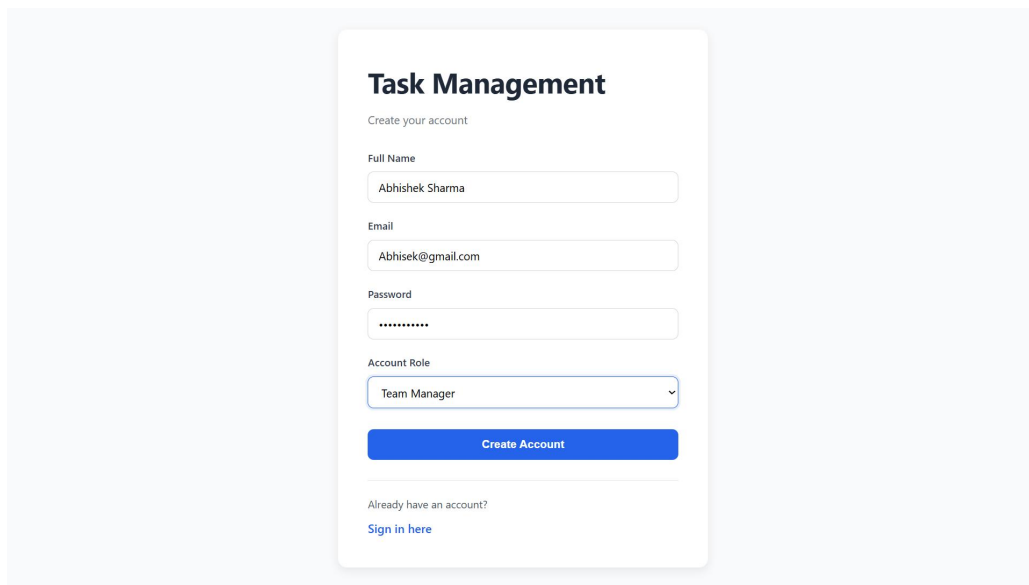
A screenshot of a web application's user registration interface. The form is titled "Task Management" and "Create your account". It contains input fields for "Full Name" (filled with "Abhishek Sharma"), "Email" (filled with "Abhishek@gmail.com"), and "Password" (filled with "*****"). There is a dropdown menu for "Account Role" with "Team Manager" selected. A blue "Create Account" button is at the bottom. Below the button, there is a link "Sign in here" for users who already have an account.

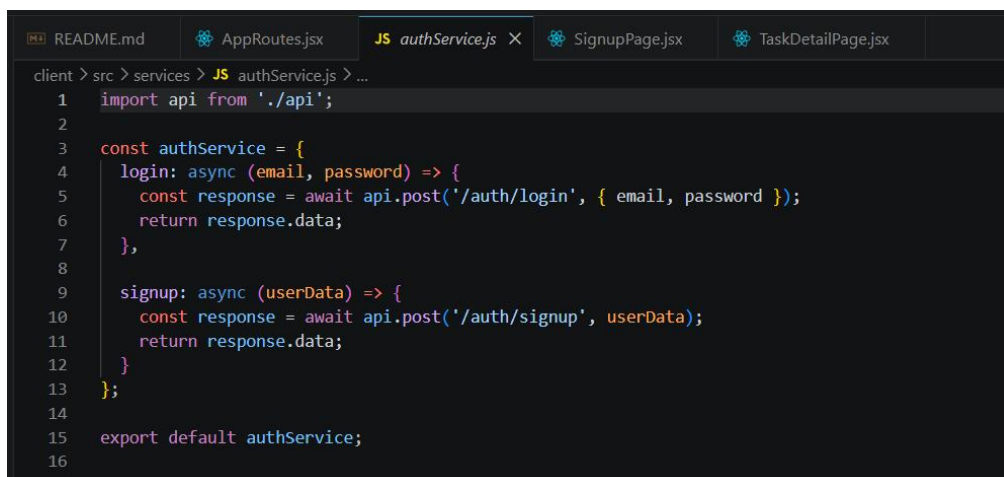
Figure 2.2 — User registration interface implementation.

Authentication Service Integration

- Implemented authentication service methods using the centralized Axios instance.

Configured services:

- User Login
 - User Registration
- Established communication with backend authentication endpoints.

A screenshot of a code editor showing the implementation of an authentication service in a file named `authService.js`. The code uses the `axios` library to make API calls. It defines two asynchronous functions: `login` and `signup`. The `login` function takes `email` and `password` as arguments and calls `api.post('/auth/login', { email, password })`. The `signup` function takes `userData` as an argument and calls `api.post('/auth/signup', userData)`. Both functions return the `response.data`. The code is wrapped in a `const authService = { ... }` object and exported as the default export.

```
1 import api from './api';
2
3 const authService = {
4   login: async (email, password) => {
5     const response = await api.post('/auth/login', { email, password });
6     return response.data;
7   },
8
9   signup: async (userData) => {
10    const response = await api.post('/auth/signup', userData);
11    return response.data;
12  }
13 };
14
15 export default authService;
```

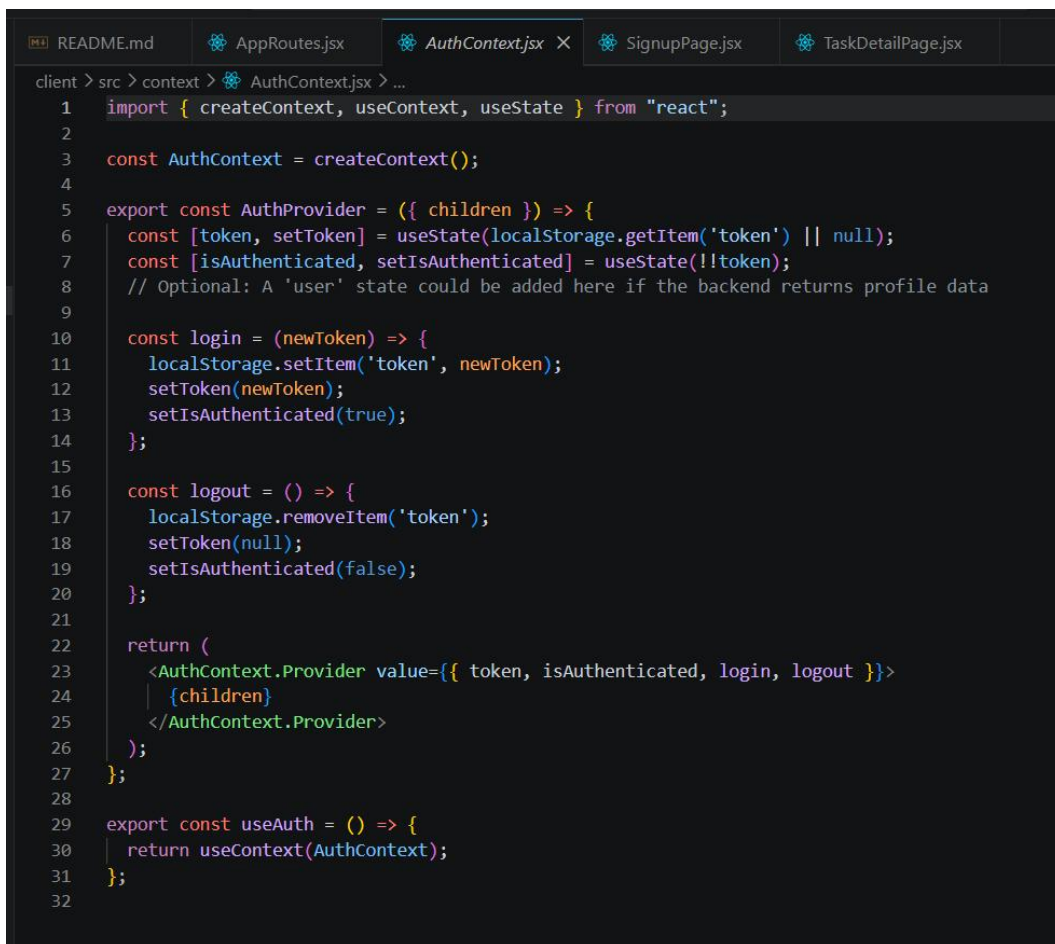
Figure 2.3 — Authentication service integrated with backend APIs.

Authentication Context Implementation

- Created a global authentication context using React Context API.

Implemented functionality:

- Authentication state management
 - Token persistence
 - Login handling
 - Logout handling
 - Authentication status tracking
- Added reusable useAuth hook for future application modules.



```
client > src > context > AuthContext.jsx > ...
1  import { createContext, useContext, useState } from "react";
2
3  const AuthContext = createContext();
4
5  export const AuthProvider = ({ children }) => {
6    const [token, setToken] = useState(localStorage.getItem('token') || null);
7    const [isAuthenticated, setIsAuthenticated] = useState(!token);
8    // Optional: A 'user' state could be added here if the backend returns profile data
9
10   const login = (newToken) => {
11     localStorage.setItem('token', newToken);
12     setToken(newToken);
13     setIsAuthenticated(true);
14   };
15
16   const logout = () => {
17     localStorage.removeItem('token');
18     setToken(null);
19     setIsAuthenticated(false);
20   };
21
22   return (
23     <AuthContext.Provider value={{ token, isAuthenticated, login, logout }}>
24       {children}
25     </AuthContext.Provider>
26   );
27 };
28
29 export const useAuth = () => {
30   return useContext(AuthContext);
31 };
32
```

Figure 2.4 — Global authentication context implementation.

JWT Token Management

- Configured JWT token storage using browser local storage.

Implemented functionality:

- Store token after successful login
 - Restore token on application refresh
 - Remove token during logout
-
- Ensured authentication state remains available across page reloads.

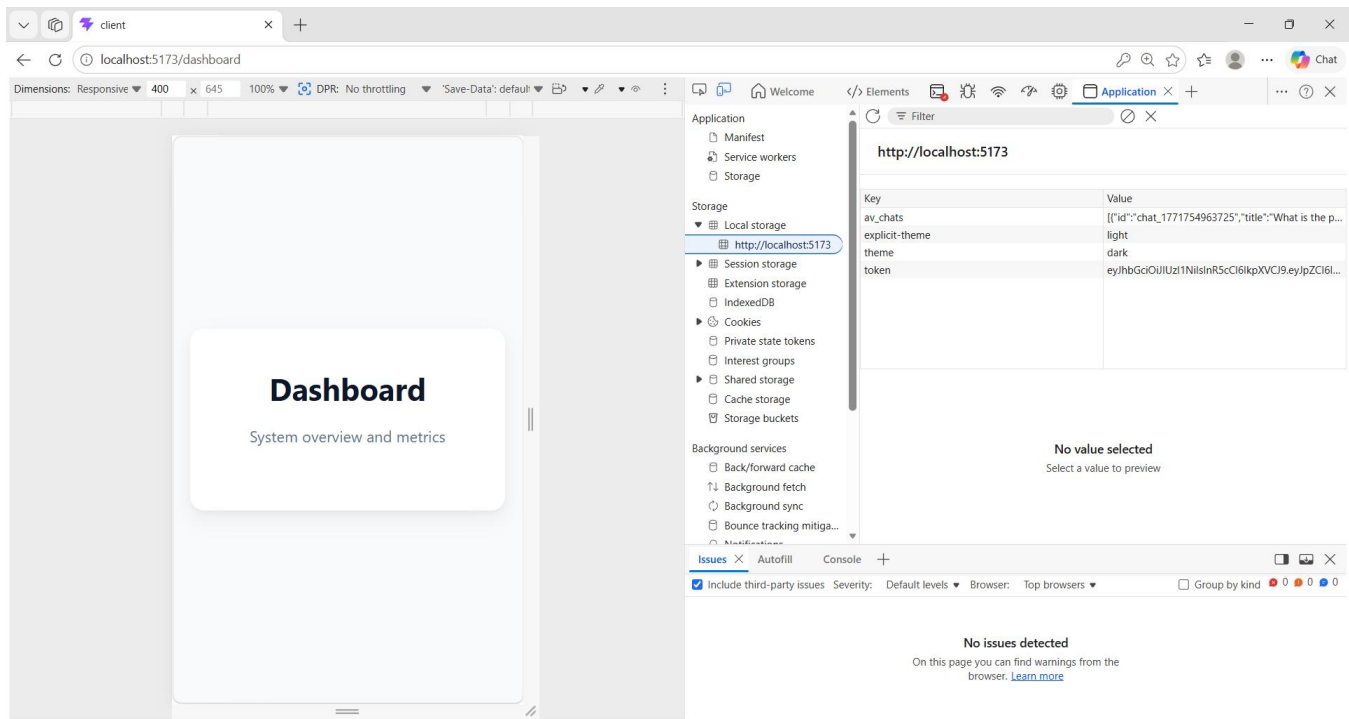


Figure 2.5 — JWT token stored in browser local storage after authentication.

Authentication Workflow Verification

- Verified complete authentication flow from frontend to backend.

Validated process:

- User login request
- Backend authentication
- Token storage
- Authentication state update
- Dashboard redirection

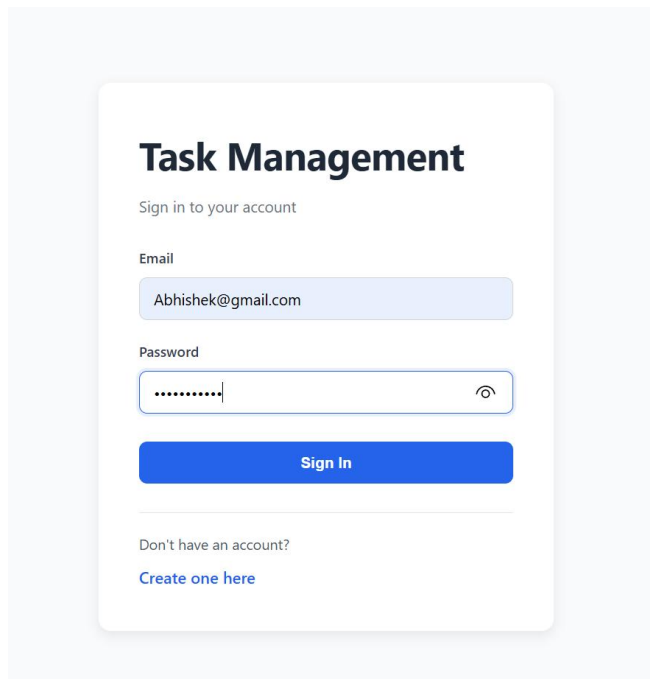


Figure 2.6 — Login interface with authentication support.

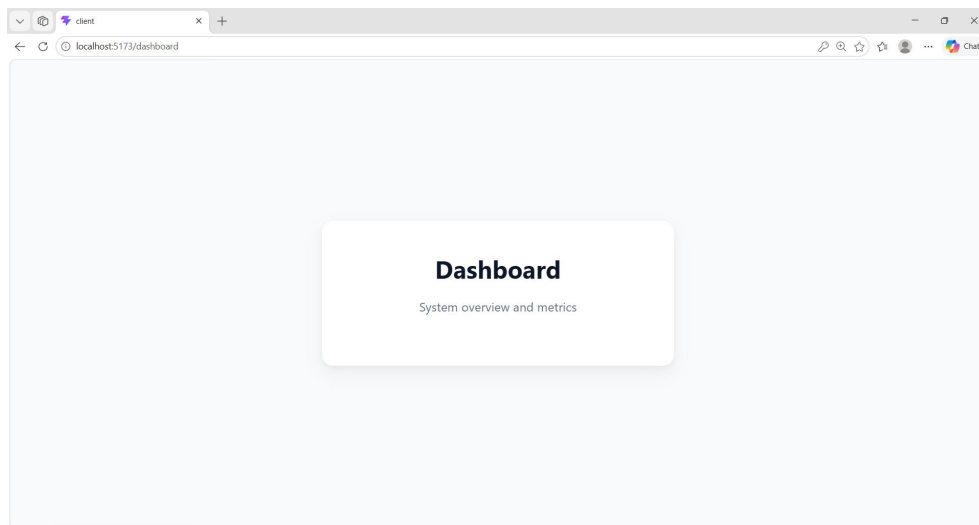


Figure 2.7 — Successful authentication and dashboard redirection.

3. Conclusion

Day 15 focused on implementing frontend authentication functionality and centralized state management. Login and registration interfaces were successfully integrated with backend APIs, JWT token management was established, and React Context was configured to maintain authentication state across the application. The project is now prepared for protected routing and authenticated user experience enhancements in the next phase.